XSS Cheat Sheet Basic and Advanced

Educational Bug Bounty Series • Authorized Security Research Only

■ XSS Cheat Sheet — Basic Concepts & Defensive Testing

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope: *Use only authorized environments and harmless markers. Do not use payloads to steal cookies, tokens, source code, or personal information.*

■ Executive Summary

Cross-Site Scripting (XSS) is an injection flaw that causes a browser to interpret untrusted input as active content. The supplied seven-page *xss.pdf* describes reflected, stored, and DOM-based XSS; input reflection contexts; multiple reflections; filenames and metadata; SVG/XML/Markdown; encoding; and WAF-specific notes.

■ Core Testing Methodology

1. Identify input sources and where their values are rendered.
2. Insert a unique harmless marker and trace it through the response and DOM.
3. Classify the context: HTML text, attribute, script string, URL, CSS, comment, or DOM sink.
4. Check whether output is encoded according to the destination context.
5. Review reflected, stored, and DOM-based flows separately.
6. Test stored content only in an authorized test account and avoid exposing it to other users.
7. Review upload filenames and metadata with synthetic files.
8. Check Markdown, SVG, XML, and other formats only where the application intentionally supports them.
9. Use scanners as supporting evidence, then manually verify findings.
10. Report a minimal reproduction and retest the fix.

■ High-Value Bug Bounty Tips & Bypasses

- **Reflected XSS:** Input is returned in the response; context and encoding determine risk.
- **Stored XSS:** Input is persisted and later rendered; test with controlled content.
- **DOM-based XSS:** Client-side JavaScript may route untrusted data into unsafe DOM sinks.
- **HTML contexts:** Comments, tags, and attributes require different handling.
- **JavaScript contexts:** String delimiters, escaping, and template construction can affect parsing; avoid executable bypass strings in public notes.
- **Multiple reflections:** One value may be inserted into more than one context.
- **File metadata:** Filenames and EXIF fields can become XSS sources if rendered unsafely.
- **SVG/XML/Markdown:** Active features differ by parser and serving context; sanitize or disable them where not required.
- **Encoding:** Repeated decoding can create inconsistencies; define canonicalization and encode at output.
- **WAF bypasses:** The source lists mixed-case, encoded, and vendor-specific variants. A WAF is not a primary fix, and these strings are intentionally omitted.
- **Source limitation:** The PDF contains many active script payloads and WAF bypasses. This guide preserves the concepts without reproducing the exploit catalogue.

■■ Educational Checklists & Safe Examples

Harmless marker:

```
XSS_TEST_12345
```

Example safe output:

```
<p>Search term: XSS_TEST_12345</p>
```

- [ ] Input source and output context are identified.
- [ ] Context-aware output encoding is applied.
- [ ] Untrusted input is not evaluated as code.
- [ ] DOM code avoids unsafe HTML-string insertion.
- [ ] Stored content is tested only with controlled accounts.
- [ ] Upload metadata and active file formats are handled safely.
- [ ] CSP is used as defense in depth.

■■ Mitigation & Remediation

Use context-aware output encoding, safe template defaults, maintained HTML sanitization libraries when HTML is a product requirement, safe DOM APIs, and a restrictive Content Security Policy. Serve untrusted files from an isolated origin and add regression tests for every confirmed reflection context.

■ Source Notes

Based on the supplied seven-page *xss.pdf*. Page 1 introduces reflected, stored, and DOM-based XSS and basic reflection contexts. Pages 2–4 cover JavaScript-string contexts, escaping, multiple reflections, comments, and file-upload filename/metadata cases. Page 5 covers SVG, Markdown, XML, encoding, and validation variants. Pages 6–7 list WAF-specific bypass examples for multiple providers. Active payloads are intentionally not reproduced.

Educational use only. Validate safely and disclose responsibly.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.